

RAG Agent



1. **Advanced RAG** — rewrite, multi-query, hybrid search, re-rank, tools, self-correct
2. **Self-RAG** — decide, retrieve, check relevance / support / usefulness
3. **CRAG (Corrective RAG)** — check answer, analyse error, re-retrieve
4. **Agentic RAG** — agent + tools + grade + rewrite (medical PDF project)
5. **Comparison table** — which one to use when

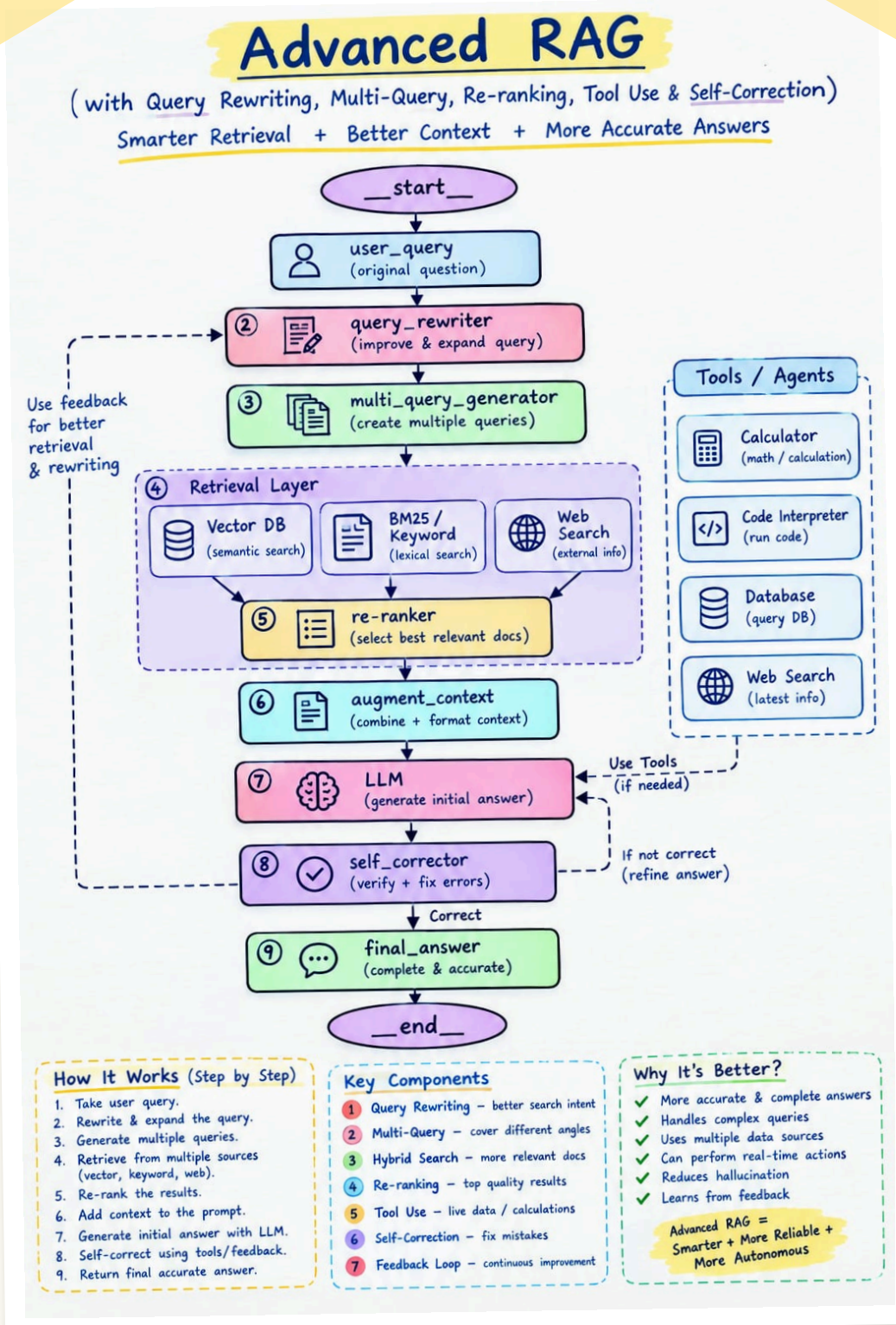
How to read each chapter:

- ① Graph picture
- ② Where we use it
- ③ Every node: **What** it does + **Why**
- ④ Advantages & Disadvantages

— Kapil Dev

1 Advanced RAG

Query Rewriting + Multi-Query + Re-ranking + Tool Use + Self-Correction



↑ Fig 1: Advanced RAG graph

★ Where / When do we use it?

- Used when a simple “search → answer” RAG is not good enough and we need **high accuracy**.
- Best for company knowledge bots, legal / medical / finance Q&A, research assistants — places where a wrong answer is costly.
- Good when user questions are vague, long, or can be asked in many ways, and data is spread over many sources (PDFs, DB, web).

Quick flow: start → user_query → query_rewriter → multi_query_generator → retrieval (Vector + BM25 + Web) → re-ranker → augment_context → LLM (+tools) → self_corrector → final_answer → end

📎 Node by Node (top → end)

1. __start__

What: Entry point of the graph. Nothing is computed here.

Why: Every LangGraph needs one fixed starting point so the flow always begins at the same place.

2. user_query

What: Takes the original question exactly as the user typed it and stores it in the graph state.

Why: We keep the original question safe so later nodes (and the self-corrector) can always compare the answer with what the user really asked.

3. query_rewriter

What: An LLM cleans and improves the question — fixes spelling, removes filler words, expands short forms (e.g. “BP” → “blood pressure”), adds missing keywords.

Why: Users write messy questions. A clear question = better search = better documents.

4.
multi_query_generator

What: Creates 3–5 different versions of the same question, each looking from a different angle.

Why: One query may miss good documents. Many queries = wider net = higher **recall** (we find more of the right chunks).

5. Retrieval Layer

What: All queries are searched in 3 places at once:

- **Vector DB** – semantic search (finds same meaning)
- **BM25 / Keyword** – lexical search (finds exact words, codes, names)
- **Web Search** – fresh, external info

Why: Each search method has blind spots. Combining them (**hybrid search**) covers each other's weakness.

6. re-ranker

What: Takes all the retrieved chunks (could be 30–50), scores each one against the question (usually a cross-encoder) and keeps only the top few.

Why: Retrieval is fast but rough. Re-ranking is slower but precise — it removes noise so the LLM sees only the best evidence.

7. augment_context

What: Combines the top chunks, removes duplicates, formats them neatly with source names, and puts them into the prompt with the question.

Why: The LLM can only use what is in its prompt. Clean, ordered context = less confusion and fewer made-up facts.

8. LLM

What: Generates the first (initial) answer from the context. If needed it can call **Tools / Agents** — Calculator, Code Interpreter, Database, Web Search.

Why: Some questions need maths, live data, or a DB lookup that text documents cannot give. Tools fill that gap.

9. self_corrector

What: Checks the answer: Is it grounded in the context? Any errors? Does it fully answer the question?

- **Correct** → go to final_answer
- **Small mistake** → back to LLM to refine
- **Bad retrieval** → feedback loop back to query_rewriter

Why: This is the quality-control gate. It catches hallucinations before the user sees them.

10. final_answer

What: Returns the verified, complete answer (often with sources).

Why: Only answers that pass the check reach the user.

11. __end__

What: Graph stops and the state is returned.

Why: Marks a clean finish of the run.

✓ Advantages

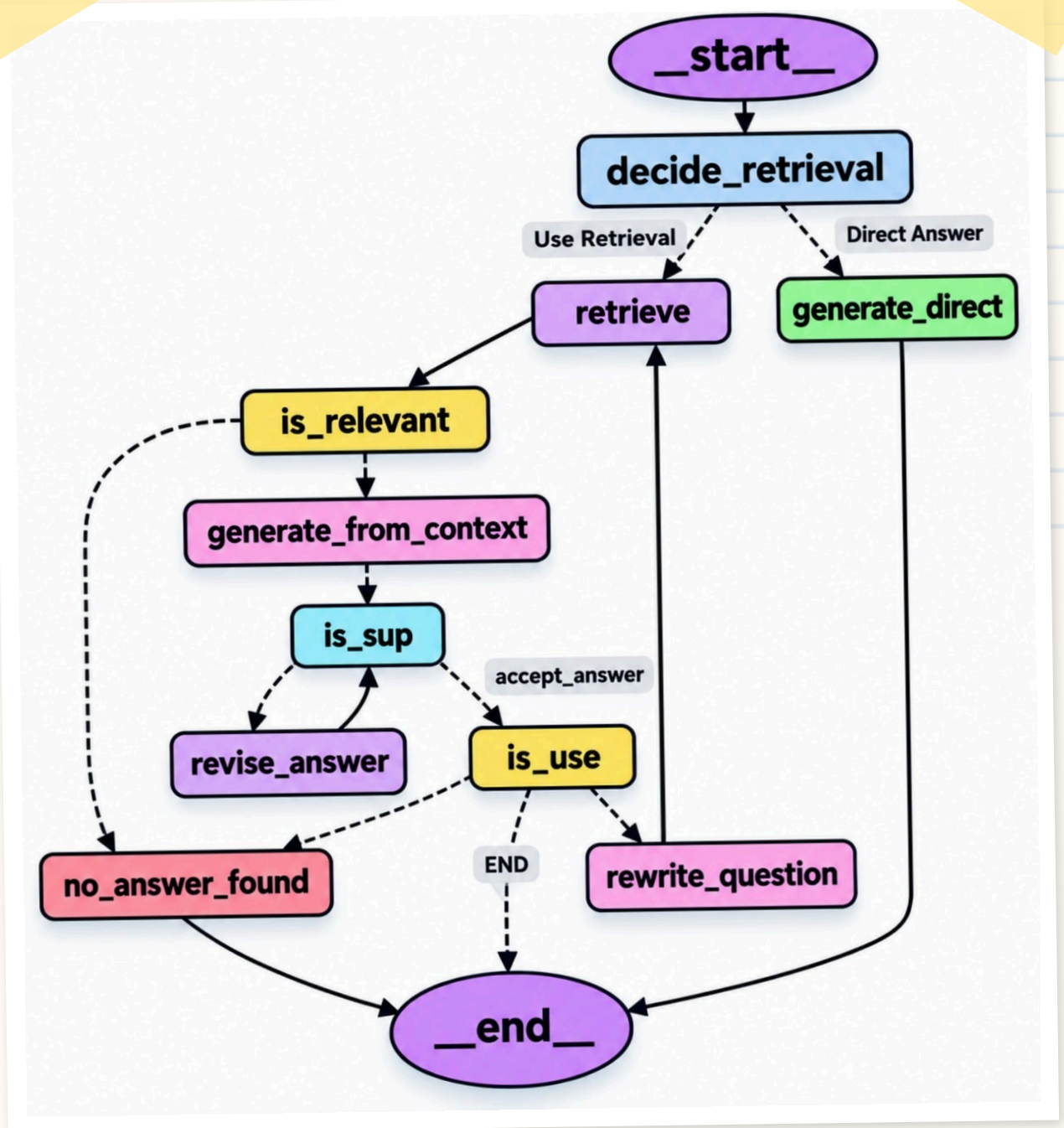
- Most accurate & complete answers of all four designs
- Hybrid search + multi-query = very high recall
- Re-ranker removes noisy chunks
- Tools give live data & correct maths
- Self-correction + feedback loop reduce hallucination
- Handles vague and complex questions well

✗ Disadvantages

- Slow — many LLM calls per question (high latency)
- Expensive — more tokens, re-ranker & web API cost
- Complex to build, debug and maintain
- Loops can run forever — must set a max-retry limit
- Overkill for simple FAQ-type questions
- Web search can bring in wrong / untrusted info

2 Self-RAG

The model decides, retrieves, then grades itself (relevance → support → usefulness)



↑ Fig 2: Self-RAG graph

★ Where / When do we use it?

- Used when **not every question needs retrieval** — e.g. “Hi” or “What is 2+2?” can be answered directly.
- Used when we must be sure every sentence of the answer is **supported by documents** (no hallucination).
- Good for chatbots that mix small talk + document questions, and for honest bots that should say “I don’t know”.

Quick flow: start → decide_retrieval → (generate_direct → end) OR (retrieve → is_relevant → generate_from_context → is_sup ⇌ revise_answer → is_use → end / rewrite_question / no_answer_found)

🔪 Node by Node (top → end)

1. __start__

What: Entry point of the graph.

Why: Fixed starting point for every run.

2. decide_retrieval

What: LLM looks at the question and decides: **Use Retrieval or Direct Answer.**

Why: Retrieval costs time and money. If the LLM already knows (greetings, general facts), skip it and save cost.

3. generate_direct

What: Answers straight from the LLM’s own knowledge, then goes to end.

Why: Fast path for easy questions — no database hit needed.

4. retrieve

What: Searches the vector store and fetches the top-k chunks for the question.

Why: Brings real evidence from our own documents.

5. is_relevant

What: Grades each retrieved chunk: relevant or not?

Irrelevant chunks are thrown away.

- some relevant → generate_from_context
- none relevant → no_answer_found

Why: Garbage in = garbage out. Bad chunks must never reach the generator.

6.
generate_from_context

What: Writes the answer using **only** the relevant chunks.

Why: Keeps the answer grounded in our data, not in the model's memory.

7. is_sup

What: Is Supported? Checks whether every claim in the answer is backed by the context.

- fully supported → accept_answer → is_use
- not supported → revise_answer

Why: This is the anti-hallucination check — catches made-up facts.

8. revise_answer

What: Rewrites the answer, removing or fixing unsupported lines, then sends it back to is_sup for another check.

Why: Fixing is cheaper than starting again. The loop continues until the answer is supported (or a retry limit is hit).

9. is_use

What: Is Useful? Checks whether the answer actually solves the user's question.

- useful → END
- not useful but can try again → rewrite_question
- not useful and retries finished → no_answer_found

Why: An answer can be true but still useless (off-topic). This node checks helpfulness.

10. rewrite_question

What: Rephrases the question and jumps back to retrieve for a fresh search.

Why: The first search may have used the wrong words; a better query finds better chunks.

11. no_answer_found

What: Returns an honest message like "I couldn't find this in the documents."

Why: Saying "I don't know" is far safer than giving a confident wrong answer.

12. __end__

What: Graph stops and returns the final state.

Why: Clean finish.

✓ Advantages

- Saves cost — skips retrieval when not needed
- Three checks: relevance, support, usefulness
- Very low hallucination — answers are grounded
- Honest "no answer found" path
- Self-healing loops (revise + rewrite)

✗ Disadvantages

- Many grader LLM calls → slower and costlier
- decide_retrieval can be wrong (skips retrieval when needed)
- Graders themselves can make mistakes
- Only searches our own docs (no web fallback)
- Needs retry limits or loops never end
- Harder to tune prompts for each grader

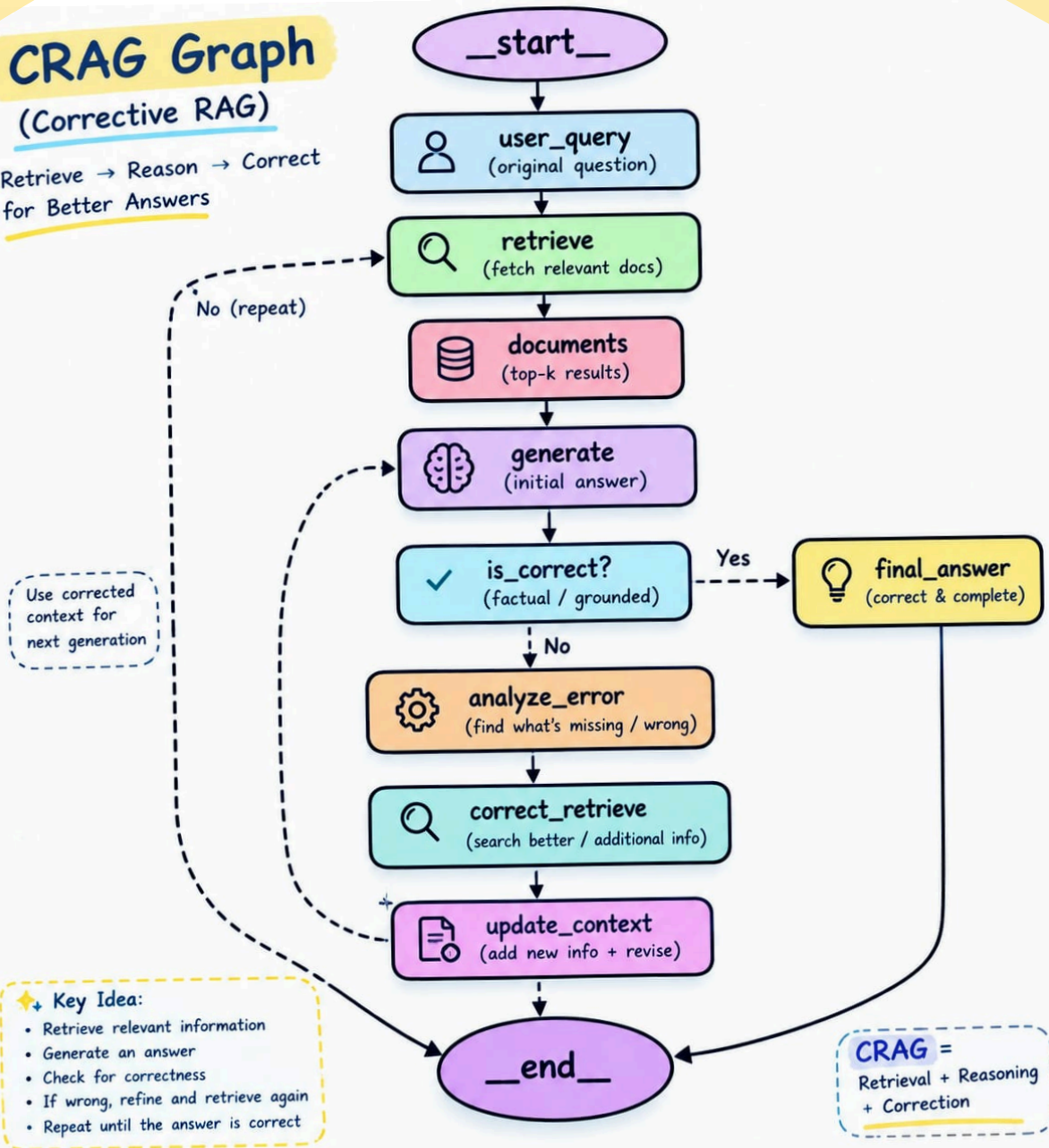
3

CRAG – Corrective RAG

Retrieve → Reason → Correct, for better answers

CRAG Graph (Corrective RAG)

Retrieve → Reason → Correct
for Better Answers



↑ Fig 3: CRAG – Corrective RAG graph

★ Where / When do we use it?

- Used when our retriever is **not always reliable** and sometimes brings wrong or incomplete documents.
- Used when the answer must be **factually correct** and we are ready to search again to fix it.
- Good for knowledge bases with gaps, where extra / external search (web) can fill missing info.

Quick flow: start → user_query → retrieve → documents → generate → is_correct? → Yes: final_answer → end | No: analyze_error → correct_retrieve → update_context → (generate / retrieve / end)

📌 Node by Node (top → end)

1. __start__

What: Entry point.

Why: Fixed start.

2. user_query

What: Receives the original question and saves it in state.

Why: The checker later needs the original question to judge correctness.

3. retrieve

What: Searches the knowledge base for relevant documents.

Why: First attempt to collect evidence.

4. documents

What: Holds the top-k results returned by the retriever.

Why: Keeps the evidence in state so that generate and the checker both use the same set.

5. generate

What: LLM writes an initial answer from the documents.

Why: We need a draft first before we can check it.

6. is_correct?

What: Checks if the answer is **factual and grounded** in the documents.

- Yes → final_answer
- No → analyze_error

Why: The main gate of CRAG — decides whether correction is needed.

7. final_answer

What: Returns the correct & complete answer, then goes to end.

Why: Only checked answers are delivered.

8. analyze_error

What: Finds out **what** is wrong or missing — wrong fact? missing detail? documents off-topic?

Why: You can't fix a problem without knowing it. This makes the next search targeted, not random.

9. correct_retrieve

What: Runs a better / additional search based on the error — new keywords, different source, or web search.

Why: Brings the missing or correct information.

10. update_context

What: Adds the new info to the context, drops wrong docs, and revises.

- → back to generate (use corrected context)
- → back to retrieve ("No, repeat" if still weak)
- → end (when limit reached)

Why: Fresh, cleaned context gives the next generation a real chance of being right.

11. __end__

What: Graph finishes.

Why: Clean finish.

✓ Advantages

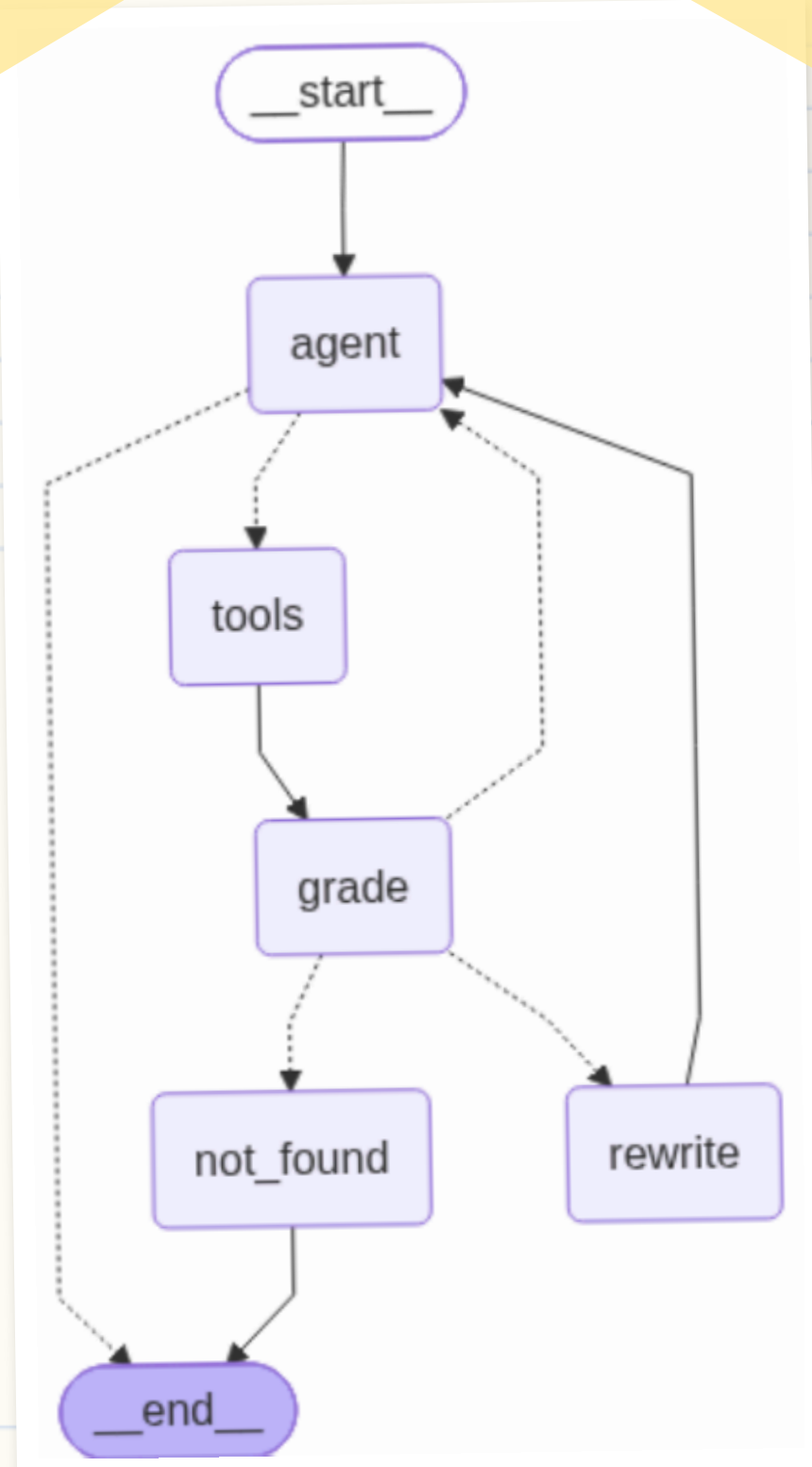
- Fixes bad retrieval instead of trusting it blindly
- Targeted correction (knows what went wrong)
- Can pull in external / web info to fill gaps
- Much more factual answers than naive RAG
- Simple, easy-to-follow linear flow + loops

✗ Disadvantages

- Extra LLM calls for checking & analysing
- Higher latency when correction loop runs
- Checker may wrongly accept a bad answer
- External/web data can add noise
- No “direct answer” path — always retrieves
- Needs a max-loop limit to stop

4 Agentic RAG

My medical project – LangGraph: agent → tools → grade → rewrite / not_found – built over text_medical.pdf + ChromaDB



↑ Fig 4: Agentic RAG graph

★ Where / When do we use it?

- Used when we want the **LLM itself (agent)** to decide whether to search, which tool to call, and when to stop.
- In this project: a **medical Q&A bot** over `text_medical.pdf`, stored in `chroma_db_agent`, with chat memory saved in `agent_memory.db` (SQLite checkpointer).
- Good for chat assistants that mix normal talk + document questions, and can grow to many tools later.

Quick flow: start → agent → (end OR tools → grade) → grade: relevant → agent | not relevant → rewrite → agent | give up → not_found → end

📌 Node by Node (top → end)

1. `__start__`

What: Entry point. The user message (+ previous chat from memory) enters the state.

Why: Fixed start; the checkpointer loads earlier conversation using the `thread_id`.

2. agent

What: An LLM with tools bound (e.g. a **retriever tool**). It reads the messages and decides:

- call a tool → **tools**
- already can answer → **__end__** (dotted edge)

Why: The brain of the graph. It avoids useless retrieval for small talk and chooses the right tool for real questions.

3. tools

What: Executes the tool call — searches ChromaDB and returns the matching chunks from the medical PDF as a `ToolMessage`.

Why: Gives the agent real evidence from our document instead of its own memory.

4. grade

What: Grades the retrieved chunks against the question (yes / no relevance). Routes:

- relevant → back to **agent** to write the final answer
- not relevant → **rewrite**
- still not relevant after max tries → **not_found**

Why: In medical use a wrong answer is dangerous. Only relevant evidence is allowed into the answer.

5. rewrite

What: Rephrases the question (better medical terms, clearer intent) and sends it back to the agent to search again.

Why: The first query might not match the words in the PDF; a better query finds the right chunks.

6. not_found

What: Returns a safe message: "This information is not available in the document."

Why: Stops endless loops and prevents made-up medical advice.

7. __end__

What: Final answer is returned and the state is saved to memory (agent_memory.db).

Why: Clean finish + the next question can use chat history.

✓ Advantages

- Agent decides when to retrieve — flexible & efficient
- Easy to add more tools later (web, calculator, DB)
- grade node keeps answers relevant & safe
- rewrite loop improves weak queries
- not_found gives an honest safe exit
- Persistent memory = real multi-turn chat

✗ Disadvantages

- Agent may skip retrieval and answer from its own memory
- No explicit hallucination check on the final answer
- Tool-calling depends on a strong LLM
- Loops add latency & cost — need retry counter
- Harder to predict / debug than a fixed pipeline
- Only one PDF source — limited coverage

5

Quick Comparison

	Advanced RAG	Self-RAG	CRAG	Agentic RAG
Main idea	Better query + better docs + self-check	Model decides & grades itself 3 times	Check answer, fix retrieval	LLM agent picks tools
Skips retrieval?	No	Yes (decide_retrieval)	No	Yes (agent → end)
Checks docs?	re-ranker	is_relevant	via is_correct	grade
Checks answer?	self_corrector	is_sup + is_use	is_correct	— (no)
Fix loop	refine / rewrite query	revise_answer, rewrite_question	analyze_error → correct_retrieve	rewrite → agent
"I don't know" exit	—	no_answer_found	—	not_found
Speed / Cost	Slowest / highest	Medium-high	Medium	Low-medium
Best for	Max accuracy, many sources	Mixed chat + strict grounding	Unreliable retriever / KB gaps	Flexible assistants, many tools

★ **Remember:** All four are the same idea — retrieve → generate — plus different checks and loops. More checks = more accuracy but more time & money. Always add a **max-retry counter** in the state so no loop runs forever.

Exam tip: Self-RAG grades itself (relevance, support, usefulness). CRAG corrects the retrieval. Agentic RAG lets the agent choose. Advanced RAG improves every stage of the pipeline.